Collapse-Aware Maintenance of Encrypted Traffic Classifiers under Temporal Drift

***Abstract*—Traffic-based inference supports device management and security monitoring in Internet of Things (IoT) networks, where evolving traffic calls for reliable classifier maintenance. We examine longitudinal encrypted application traffic, where aggregate metrics can conceal severe class-specific failures. We formalize absorber collapse, a class-asymmetric failure in which dominant classes absorb predictions from victim classes and drive their recall toward zero. On CESNET-TLS-Year22, 12 of 178 classes collapse within nine months, while five tested traffic-adapted test-time adaptation methods leave collapse-class F1 below 0.031. We propose Collapse-Aware Repair (CARE), which combines unsupervised class-health monitoring with head-only repair using newly labeled target samples, all-class reference replay, and knowledge distillation. With Batch Active Learning by Diverse Gradient Embeddings (BADGE), CARE raises macro-F1 from 0.629 to 0.700 and collapse-class F1 from 0.028 to 0.332 using 1,000 target labels under strict query-excluded evaluation over five repair seeds, while retaining stable-class F1 near the static level (0.894 versus 0.903). A QUIC stress test probes cross-protocol transfer. A separate 11-household IoT device-type holdout shows low-recall-class recovery alongside heterogeneous gains and losses across homes, underscoring the need to review candidate updates at the class level.**

***Index Terms*—Encrypted traffic classification, IoT traffic analytics, concept drift, class collapse, label-efficient maintenance, active learning.**

I. Introduction

Traffic-based inference supports device inventory and security monitoring in IoT networks, even when traffic payloads are unavailable. Longitudinal studies of residential IoT networks document temporal and cross-home drift in device-type inference [1]; subsequent work examines class-prior adaptation [2] and drift-dependent labeling budgets [3]. These findings motivate a broader maintenance question for traffic classifiers: can aggregate performance conceal the failure of individual classes, and can a limited number of fresh labels repair them without damaging stable classes? We study this question in longitudinal encrypted application traffic and examine its reach in a separate 11-household IoT device-type setting.

Deep learning has become a common approach for encrypted application traffic classification, achieving high accuracy on benchmark datasets [4], [5], [6], [7]. However, a growing body of evidence shows that these classifiers degrade rapidly after deployment. Prior studies report frequent class drift on CESNET-TLS-Year22 [8] and substantial F1 degradation over time on Internet service provider (ISP)-scale data [9]. The causes are diverse: application updates alter protocol behavior, Transport Layer Security (TLS) certificate rotations change handshake fingerprints, and content delivery network (CDN) migrations shift Internet Protocol (IP) ranges and packet-size distributions [8], [10], [11].

For a deployed traffic classifier, acceptable aggregate performance does not ensure that every class remains usable. Encrypted-traffic classification illustrates this reliability problem: predictions can support quality-of-service monitoring, anomaly triage, capacity planning, and policy enforcement [12], [6], yet a collapsed class could be systematically omitted from the resulting decisions. This motivates class-aware maintenance that explicitly measures class recovery alongside preservation of the remaining classes under a limited labeling budget.

Existing approaches to this problem span three categories. Label-free self-evolving adaptation uses high-confidence pseudo-labels to update a model without human annotation [13], but its behavior on collapsed classes has not been established. Unsupervised test-time adaptation (TTA) from computer vision [14], [15], [16] avoids labeling but does not repair collapsed classes under our tested traffic adaptations. Standard active learning [17], [18], [19] can select informative samples, but generic strategies are not designed for the specific failure mode we identify. This motivates *class-aware, label-efficient maintenance*: diagnosing collapse and assessing whether a small supervised update recovers affected classes while preserving the rest.

We formalize and systematically characterize a previously underexamined class-asymmetric failure pattern that we term the *absorber-collapse pattern*. Our systematic confusion analysis on the CESNET-TLS-Year22 dataset [20] shows that performance degradation is not uniform across classes. Instead, a small number of dominant "absorber" classes systematically attract predictions away from "victim" classes, causing the victims to collapse to near-zero recall within a short deployment period. Traffic-adapted TTA variants, including SAR [16] and NOTE [21], do not materially recover these collapsed classes under our implementations.

In response to these limitations, we propose CARE (Collapse-Aware REpair), a collapse-aware, operator-in-the-loop maintenance framework for encrypted-traffic classifiers. Unlike unsupervised TTA, which adapts without collapse-specific supervision, and standard active learning, which selects informative samples without explicitly evaluating class-collapse recovery, CARE couples class-health diagnosis with a repair pipeline assessed for both collapse recovery and stable-class preservation.

CARE shifts maintenance from confidence-driven, label-free adaptation to targeted, label-efficient repair. A multi-signature unsupervised monitor ranks class-level risk for operator review. Once a repair cycle is approved, Margin or BADGE selects a small batch of informative target samples for labeling; CARE then updates only the classification head using the labeled target features, all-class reference-period replay,

and knowledge distillation. This design addresses two linked objectives: recovering collapsed classes and limiting forgetting across the remaining stable classes.

CARE supports scheduled, operator-reviewed maintenance. The deployed model continues serving while an operator reviews monitoring alerts, approves label acquisition and repair, and validates a candidate head before activation; the original model can be retained for rollback. Freezing the encoder bounds the update and avoids full-model retraining.

Our evaluation is designed to test both repair effectiveness and its operating limits. The main experiments use strict query exclusion, repeated runs, collapse/stable F1, and per-class recovery status; additional matched-holdout experiments compare CARE with a Chen-inspired self-evolving baseline, while broader comparisons cover TTA variants, standard active-learning selectors, and adapted continual-learning or drift methods. Further studies examine different drift periods, model backbones, a QUIC cross-protocol stress test, and an 11-household IoT device-type task to assess the framework's operating range.

Together, these results assess class recovery and preservation within one diagnosis-and-repair procedure.

The study makes three contributions to class-aware maintenance.

First, we formalize absorber collapse and provide longitudinal, per-class evidence together with two descriptive Fisher signatures.

Second, we develop and evaluate a label-efficient repair framework that integrates diagnostic monitoring, active target-label selection, all-class reference replay, and head-only updating in an operator-reviewed offline workflow. It organizes established learning components around class-level recovery and preservation in a controlled maintenance cycle.

Third, we evaluate class recovery and stable-class damage with strict query exclusion, matched holdouts, comparative baselines, multiple drift periods and backbones, and a QUIC cross-protocol stress test. A separate 11-household IoT evaluation quantifies device-type recovery and variation in repair outcomes across homes.

II. Related Work

Traffic-based IoT device inference faces temporal and cross-home concept drift. Pashamokhtari et al. [1] studied model selection across residential networks, Azizi et al. [2] examined adaptation to changing device-class proportions, and a recent preprint [3] investigated drift-dependent labeling budgets and uniform traffic sampling. These studies establish temporal drift and limited labeling as practical challenges for IoT traffic inference. CARE addresses a complementary maintenance question: how to recognize severe class-level degradation and repair known classes with scarce labels while preserving the others.

Encrypted traffic classifiers are intrinsically non-stationary after deployment because the packet-sequence statistics on which they rely change with applications, protocols, and network infrastructure [12], [8], [9]. Longitudinal studies have documented the resulting degradation: Soukup et al. [8] identified 28 frequently drifting classes on CESNET-TLS-Year22; Malekghaini et al. [9] observed that "web is the class most of the misclassified flows are confused with"; and Jerabek et al. [22] showed that random splitting inflates performance by at least 3.5 percentage points relative to time-based evaluation. Yet prior work has largely pursued two broad remedies—label-free adaptation and label-assisted maintenance—without formulating the high-confidence, class-asymmetric absorber-collapse failure studied here. This omission matters because acceptable aggregate performance can coexist with near-zero recall for a small set of victim classes.

Label-free test-time adaptation (TTA) includes Tent [14], EATA [15], SAR [16], CoTTA [23], and NOTE [21]. Tent minimizes prediction entropy, while the other methods add different sample-selection, optimization, or teacher-update mechanisms for broader target-domain adaptation. The key mismatch for absorber collapse lies in the adaptation signal: when an absorber confidently captures a victim's predictions, entropy is low, confidence is high, and a pseudo-label reproduces the same error. Confidence- or pseudo-label-driven adaptation therefore has little corrective signal for that victim and may propagate the error instead. Consistent with this mechanism, none of the tested TTA variants materially improves collapse F1 (§III-E), and the limitation persists with both GroupNorm and BatchNorm variants (§V-D1). Chen et al. [13] likewise used class-aware drift triggering and high-confidence pseudo-labels for full-model self-evolution, but did not report collapse-specific recovery metrics. These methods remain relevant to general distribution shift, but they are not designed to diagnose and repair confidently misclassified absorber–victim pairs.

A second family adapts models through domain alignment, replay, or supervised updates. Domain-Adversarial Neural Networks (DANN) [24] and Conditional Domain Adversarial Networks (CDAN) [25] use global alignment objectives that do not explicitly represent class-specific collapse and may mix victim and absorber regions under strongly class-asymmetric shift. Traffic-oriented continual-learning methods such as ILETC [26], MEMENTO [27], CPN [28], and PRIME [29], together with general preservation methods such as EWC [30] and LwF [31], provide useful replay, distillation, or regularization mechanisms. However, they primarily address new classes or tasks and catastrophic forgetting [32]. The present setting instead retains a fixed label space and seeks to recover existing classes collapsed by temporal drift while limiting damage to stable classes. It also differs from static long-tailed classification [33], [34], because absorber–victim confusion is induced by temporal change rather than by a fixed class-frequency distribution.

Active learning provides a direct way to allocate a limited target-label budget, but its selection objective matters. Generic entropy and coreset strategies prioritize global uncertainty or diversity [17], [18], whereas margin sampling and BADGE [19] deliberately favor low-margin or diverse uncertain samples and can therefore include relevant decision boundaries. Adjacent traffic studies address only parts of the problem: CADE [35] targets novel-class drift detection, Liu et al. [36] considered drift-aware active learning more broadly, and NetGuard [37] studied generative active adaptation for drifting and imbalanced intrusion detection. Their tasks and

label spaces differ from fixed-class encrypted-application repair, so we treat them as adjacent rather than budget-matched baselines. In our controlled comparison, entropy and coreset substantially trail random, margin, and BADGE under the same replay and distillation pipeline (§V-C2). This result indicates that label selection is consequential, but selection alone does not supply collapse diagnosis or stable-class preservation.

The reviewed studies motivate three aspects of the present investigation: explicitly characterizing absorber collapse, assessing recovery together with damage to stable classes under a small label budget, and evaluating bounded head repair using query-excluded and matched-holdout protocols. CARE integrates established monitoring, active-learning, replay, and distillation components into a class-level maintenance workflow evaluated across temporal and protocol shifts.

III. Empirical Analysis: Class Collapse Under Drift

This section first formalizes absorber collapse, then describes the longitudinal evaluation setting, characterizes the observed absorber–victim failure pattern and its empirical signatures, and finally examines why general test-time adaptation methods do not repair it.

A. Problem Formulation

Let $f_\phi : \mathcal{X} \to \mathbb{R}^d$ denote a frozen feature encoder and $h_\theta : \mathbb{R}^d \to \mathbb{R}^C$ a classification head, jointly trained on a source period $\mathcal{D}^{\text{src}}$ with C application classes. At a later target period t, the data distribution shifts: $P_t(\mathbf{x}, y) \neq P_{\text{src}}(\mathbf{x}, y)$. We define a class c as *collapsed* at period t if its recall $R_c^{(t)} = \text{TP}_c/(\text{TP}_c + \text{FN}_c) < \tau$, where TP and FN denote true positives and false negatives; we use τ=0.1. A collapsed class c has a *primary absorber* $a(c) = \arg\max_{j \neq c} n_{c \to j}$, where $n_{c \to j}$ counts samples with true label c predicted as j. The goal is to update h_θ using a small label budget $B \ll |\mathcal{D}^{\text{tgt}}|$ and optional reference-period replay, recovering collapsed classes while preserving stable-class performance.

B. Longitudinal Analysis Setup

Most public encrypted-traffic datasets are static snapshots without temporal partitions, making them unsuitable for studying concept drift over time. CESNET-TLS-Year22 [20] is one of the few datasets that provides month-level temporal splits spanning an entire year, which is essential for our longitudinal collapse analysis. It contains 507.7M TLS flows across 12 months of 2022 from a university network. Its encrypted application labels provide a longitudinal test of class-level repair; the separate household study examines IoT device types. We use the `size=S` split provided by the CESNET DataZoo application programming interface (API). The M12 target pool used by the strict and matched-holdout experiments contains 1,323,229 instances across 178 application classes; monthly counts vary with the dataset partition. Each flow is represented by Per-Packet Information (PPI): packet sizes, directions, and inter-packet times for the first 30 packets.

We train a convolutional neural network (CNN) classifier (3-layer 1D convolution with GroupNorm, 8 groups, 256-dim hidden layer, 178 classes) on March 2022 alone (M3) and evaluate on M4–M12, giving a minimum time gap of one month and maximum of nine months. Training uses a joint classification + self-supervised learning (SSL) objective: cross-entropy loss plus three auxiliary SSL tasks (masked packet-feature prediction, packet-order prediction, and flow-statistics reconstruction) with SSL weight 1.0, mask ratio 0.15, and auxiliary loss weights α=0.2, β=0.1. The SSL tasks improve encoder generalization but are not used during CARE repair (which freezes the encoder and updates only the classification head). Training runs for 30 epochs with AdamW [38] (learning rate 10^{-3}, weight decay 10^{-4}), batch size 1024, using the DataZoo default validation split.

C. The Absorber-Collapse Pattern

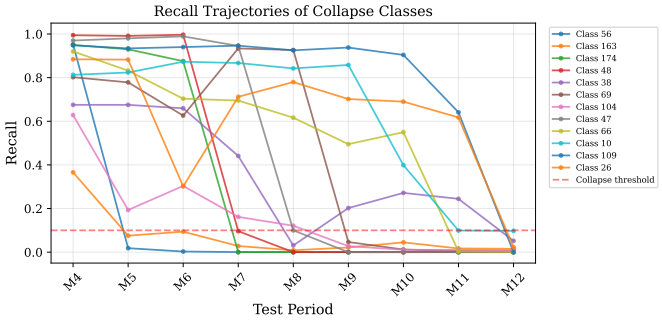


Fig. 1. Recall trajectories of 12 collapse classes on CESNET-TLS-Year22. Classes exhibit both abrupt collapse (*e.g.*, class 56 at M5) and gradual decline (*e.g.*, class 104 over M4–M12). By M12, all 12 classes fall below the 0.1 recall threshold (dashed line).

Figure 1 shows the recall trajectories of 12 classes that collapse below 0.1 recall by month 12. The collapses follow two distinct temporal patterns:

- **Abrupt collapse**: Recall drops sharply within one month (*e.g.*, class 56 drops from 0.952 at M4 to 0.018 at M5).
- **Gradual collapse**: Recall decays over several months (*e.g.*, class 104 declines from 0.629 at M4 to 0.028 at M9 and 0.010 by M12).

Crucially, these collapses are not random—each victim class has a *primary absorber*: the dominant confusion target that receives the largest share of the victim's mispredictions. Table I lists the 12 collapse classes and their top absorbers, with confusion rates ranging from 13.9% to 46.1%.

D. Two Empirical Signatures of Collapse

For descriptive analysis, we compare the Fisher discriminant ratio (inter-class distance divided by the sum of intra-class spreads) between victim–absorber pairs at M4 and M12. We group classes using heuristic thresholds on the relative change $\Delta J = (J_{\text{M12}} - J_{\text{M4}})/J_{\text{M4}}$: a *reduced-separability signature* when $\Delta J < -10\%$ and a *boundary-mismatch signature* when $\Delta J > +20\%$. These groups are empirical signatures rather than identified causal mechanisms:

- **Reduced-separability signature** (9 of 12 classes, including 56, 174, 104, and 109): The Fisher ratio decreases by

TABLE I
ABSORBER-COLLAPSE PAIRS AT M-2022-12. EACH COLLAPSED CLASS HAS A DOMINANT CONFUSION TARGET (ABSORBER). CONFUSION RATE = FRACTION OF VICTIM SAMPLES PREDICTED AS THE ABSORBER.

Victim Class	Absorber Class	Confusion Rate	Victim Recall	Pattern Type	First Collapse
56	96	25.2%	0.000	abrupt	M5
163	46	41.8%	0.015	gradual	M5
174	2	29.2%	0.000	abrupt	M7
48	14	39.0%	0.001	abrupt	M7
38	45	46.1%	0.052	abrupt	M8
69	105	37.7%	0.005	abrupt	M9
104	2	21.2%	0.010	gradual	M9
47	5	45.1%	0.000	gradual	M11
66	71	13.9%	0.001	abrupt	M11
10	156	36.5%	0.098	gradual	M11
109	71	24.9%	0.000	abrupt	M12
26	13	21.6%	0.022	abrupt	M12

14–59% (*e.g.*, class 109→71: 0.875 to 0.361), indicating reduced victim–absorber discriminability. The ratio alone does not distinguish centroid convergence from increased within-class spread.

- **Boundary-mismatch signature** (3 of 12 classes: 48, 69, and 10): Pairwise Fisher separation increases by 30–45%, yet the frozen classifier still absorbs the victim. This is consistent with a mismatch between the representation and the learned decision boundary; shifts in other classes are one possible, untested explanation.

The two signatures motivate targeted boundary repair with broad class preservation. CARE combines target-label fine-tuning with all-class reference-period replay, but the analysis does not assign class-level causal mechanisms.

TABLE II
DESCRIPTIVE FISHER-RATIO ANALYSIS FOR ALL 12 COLLAPSE CLASSES. J_{M4} AND J_{M12} DENOTE VICTIM–ABSORBER FISHER RATIOS, AND ΔJ IS THE RELATIVE CHANGE. THRESHOLD-DEFINED GROUPS ARE EMPIRICAL SIGNATURES, NOT CAUSAL LABELS.

Victim	Abs.	Signature	J_{M4}	J_{M12}	$\Delta J(\%)$	Static
Reduced separability ($\Delta J < -10\%$)						
56	96	red. sep.	0.886	0.627	−29	0.000
174	2	red. sep.	0.635	0.326	−49	0.000
38	45	red. sep.	0.730	0.602	−18	0.052
163	46	red. sep.	0.308	0.265	−14	0.016
104	2	red. sep.	0.431	0.361	−16	0.010
47	5	red. sep.	0.781	0.513	−34	0.000
66	71	red. sep.	0.633	0.391	−38	0.001
109	71	red. sep.	0.875	0.361	−59	0.000
26	13	red. sep.	0.367	0.188	−49	0.022
Boundary mismatch ($\Delta J > +20\%$)						
48	14	bnd. mismatch	0.561	0.730	+30	0.001
69	105	bnd. mismatch	0.293	0.426	+45	0.005
10	156	bnd. mismatch	0.435	0.567	+30	0.098

Table II quantifies the Fisher discriminant ratio trajectory for all 12 collapse classes. Reduced-separability classes (9 of 12) exhibit a mean $\Delta J = -34\%$, indicating lower victim–absorber discriminability at M12. Boundary-mismatch classes (3 of 12) show the opposite ($\Delta J \approx +35\%$): pairwise separation increases despite continued collapse, which is consistent with a frozen-boundary mismatch but does not identify its cause.

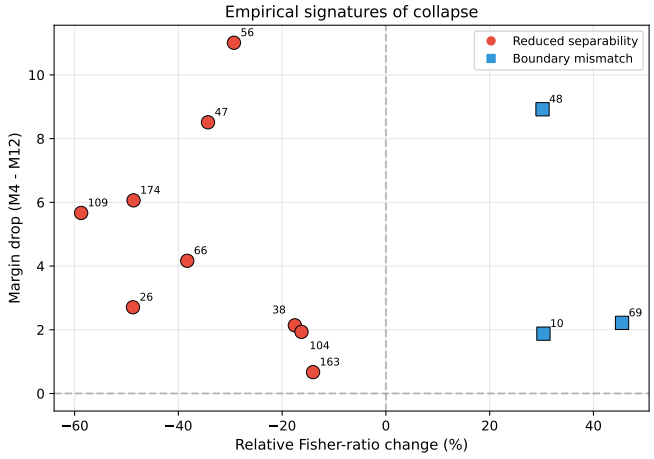


Fig. 2. Scatter plot of the two descriptive signatures for all 12 collapse classes. The x-axis shows relative Fisher-ratio change (ΔJ) from M4 to M12; the y-axis shows margin drop (M4 margin − M12 margin). Colors reproduce the threshold-defined groups. All classes show positive margin drops (0.67–11.01), and 11 of 12 exceed 1.5, consistent with reduced logit separation. Replotted from archived numeric records using a Codex-assisted Matplotlib script; the plotted values are unchanged.

Figure 2 positions each collapse class by its Fisher-ratio change and margin drop. The x-axis separation visualizes the threshold-defined grouping by construction and is not independent validation; the overlapping margin drops show that margin alone does not distinguish the two signatures.

E. Why General TTA Methods Fail

We evaluate five TTA methods from computer vision on the same setup: Tent [14], EATA [15], SAR [16], CoTTA [23], and NOTE [21]. These are traffic-adapted implementations for 1D packet-sequence inputs, with necessary simplifications: EATA uses uniform Fisher importance (rather than per-sample Fisher computed from a held-out set), CoTTA uses Gaussian noise augmentation (as no standard augmentation pipeline exists for packet sequences), and NOTE uses a simplified prediction-balanced buffer without Instance-Aware BatchNorm. Table III reports M12 results.

TABLE III
TTA BASELINES ON CESNET-TLS-YEAR22 AT M-2022-12 (TRAFFIC-ADAPTED IMPLEMENTATIONS, NO LABELS). NONE OF THE TESTED VARIANTS REPAIR COLLAPSE CLASSES UNDER OUR IMPLEMENTATIONS.

Method	Macro-F1	Collapse F1	Stable F1
Static	0.629	0.028	0.903
Tent	0.631	0.026	0.903
EATA	0.629	0.028	0.903
SAR	0.629	0.026	0.901
CoTTA	0.630	0.027	0.903
NOTE	0.630	0.027	0.902

These simplifications may underestimate the methods' potential; however, the near-zero collapse F1 across all tested variants—including Tent and SAR, which require no simplification—is consistent with a mismatch between

confidence-based unsupervised adaptation and confident absorber–victim errors. When an absorber class confidently captures a victim's predictions, entropy is *low* (the model is confidently wrong), so entropy-based adaptation sees no signal to correct. This negative result motivates the shift from unsupervised adaptation to label-efficient targeted repair.

To assess learning-rate sensitivity, we sweep five learning rates from 10^{-5} to 10^{-3} (5 configurations per method). The best collapse F1 across all configurations remains below 0.031 for every method (NOTE: 0.030, EATA: 0.028, CoTTA: 0.027, SAR: 0.027, Tent: 0.026), showing limited sensitivity to learning-rate choice over the tested range. Higher learning rates cause Tent to catastrophically degrade (macro-F1 drops from 0.63 to 0.21 at learning rate 10^{-3}).

IV. Method: Collapse-Aware Repair

CARE is a label-efficient repair framework comprising three stages: (1) unsupervised monitoring of per-class health, (2) active sample selection, and (3) head fine-tuning with all-class replay and knowledge distillation. Algorithm 1 summarizes the procedure.

Figure 3 gives the proposed operator-reviewed workflow. In a possible IoT use case, a gateway would extract flow metadata and feed a deployed device- or application-type classifier. The monitor would rank classes at risk from unlabeled predictions for operator review; an approved maintenance cycle would then proceed to label acquisition, head repair, validation, and a controlled model update. The household experiment in Section V-D4 evaluates the repair stage in a device-type setting.

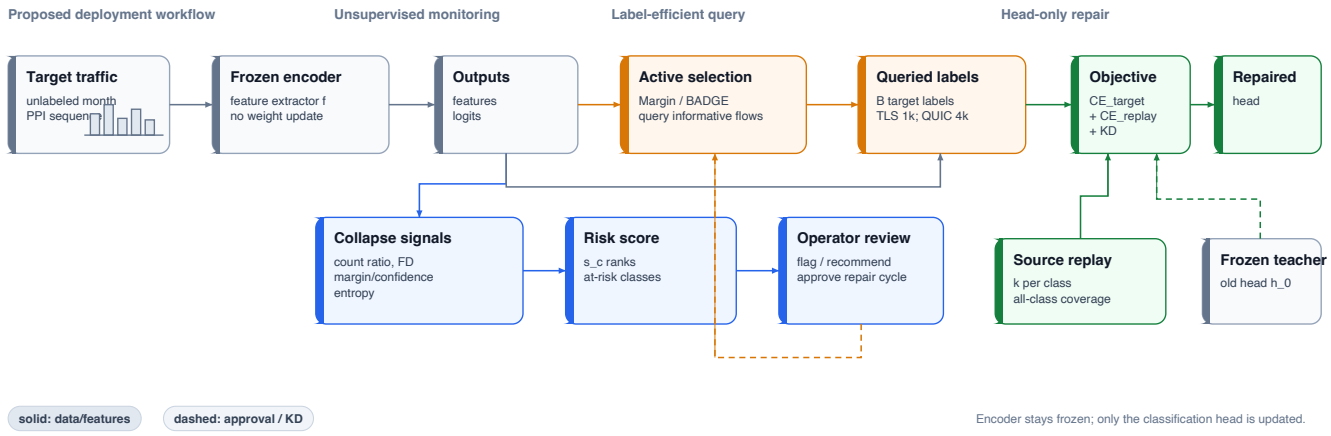


Fig. 3. Proposed operator-reviewed CARE workflow. A frozen encoder produces target-period logits and features. Monitoring ranks class-level risk and flags classes for operator review; only an approved repair cycle proceeds to active label selection and offline head-only repair with all-class reference-period replay and knowledge distillation.

A. Unsupervised Collapse Detection and Monitoring

We propose a multi-signal collapse risk score computed entirely from model predictions on unlabeled data. For each class c, we compare predictions between a reference period and the target period across five signals, all computed on samples *predicted* as class c (no true labels required):

1) **Prediction count ratio**: $r_c = (n_c^{\text{tgt}}/N^{\text{tgt}})/(n_c^{\text{ref}}/N^{\text{ref}})$. Collapsed classes lose predictions ($r_c \ll 1$).
2) **Fréchet Distance (FD)**: Feature distribution shift between periods for the predicted class region.
3) **Margin drop**: Decrease in mean prediction margin (top-2 logit gap).
4) **Confidence drop**: Decrease in mean prediction confidence.
5) **Entropy rise**: Increase in mean prediction entropy.

The collapse risk score combines these signals:

$$s_c = w_1 \cdot \max(0, 1 - r_c) + w_2 \cdot \tilde{d}_c + w_3 \cdot \tilde{m}_c + w_4 \cdot \tilde{f}_c + w_5 \cdot \tilde{e}_c \quad (1)$$

where $\tilde{d}_c$, $\tilde{m}_c$, $\tilde{f}_c$, $\tilde{e}_c$ denote the median-absolute-deviation (MAD)-normalized FD, margin drop, confidence drop, and entropy rise for class c, and $w = (0.40, 0.20, 0.15, 0.15, 0.10)$. Weights were set as a fixed heuristic based on signal informativeness (count drop is the strongest single indicator), without access to target-period labels; labels are used only retrospectively to report detection precision and recall. A sensitivity analysis (§V-E) comparing manual, equal, and historically calibrated weights shows limited variation across the three tested settings (detection F1 0.563–0.625 on M12). In the reported diagnostic, candidates must satisfy $s_c > 0.12$, after which at most the 20 highest-risk classes are retained. This is an evaluation operating point for operator review, not a validated automatic retraining gate.

This module serves two intended roles in the proposed workflow:

- **Monitoring**: It flags classes at risk of degradation for operator investigation.
- **Alerting**: Per-class risk scores inform *when* repair should be considered, without requiring labeled data to detect drift.

Importantly, the detection module does *not* guide replay class selection—all-class replay is the default (§V-C3).

B. Active Sample Selection

From the target period, we select a budget B of samples for labeling. *Margin sampling* selects globally low-margin samples, which may include absorber–victim boundaries. *BADGE* [19] selects diverse uncertain samples via gradient embeddings and achieves higher mean performance with greater algorithmic complexity than margin. Given precomputed logits and features, Margin ranks one scalar top-two gap per target instance. Our BADGE implementation forms top-three-class gradient embeddings, applies 50-dimensional principal component analysis, and performs B k-means++ seeding steps; its iterative seeding is therefore substantially more expensive than Margin's global score sort. Table V reports program-level rather than selector-only wall time.

C. Head Fine-Tuning with All-Class Reference-Period Replay

We precompute frozen features $z = f(x)$ and fine-tune only h_θ using selected target features plus k=5 labeled reference feature exemplars per class (890 total). Let D^z_{train} be the training multiset containing ρ copies of each queried target feature and one copy of each reference exemplar; the default target repeat is ρ=2:

$$\mathcal{L}_{\text{ft}} = \frac{1}{|D^z_{\text{train}}|} \sum_{(z,y) \in D^z_{\text{train}}} \ell_{\text{CE}}(h_\theta(z), y) \quad (2)$$

where ℓ_{CE} is cross-entropy loss and $|D^z_{\text{train}}| = \rho B + 890$. Thus the default B=1,000 update uses 2,890 training feature entries but only 1,000 unique newly labeled target samples. All-class replay covers all 178 classes and performs better than targeted replay in the evaluated strategy study (§V-C3).

Algorithm 1 CARE: Collapse-Aware Repair

Require: Source model f, h_{θ_0}; labeled reference features and reference predictions; unlabeled target data D_{tgt}; budget B; repeat ρ; epochs E

Monitor: Compute per-class risk scores s_c (§IV-A) from unlabeled predictions
Alert: Flag risk for operator review; proceed only after operator approval
Compute features and logits for D_{tgt}
Select B samples via margin or BADGE; query labels $\rightarrow D^z_{\text{tgt}}$
Sample D^z_{replay}: k=5 frozen features from labeled reference-period data for all 178 classes
Build D^z_{train} with ρ target copies and one reference copy
Initialize $h_\theta \leftarrow h_{\theta_0}$
for epoch $= 1, \ldots, E$ **do**
  for minibatch in a shuffled pass over D^z_{train} **do**
    Independently sample a reference minibatch for KD
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{ft}} + \lambda_{\text{KD}} \cdot \mathcal{L}_{\text{KD}}$
    Update θ via AdamW on $\mathcal{L}$
  end for
end for
return Updated classifier h_θ

D. Knowledge Distillation

On replay feature exemplars, we add a KL-divergence loss [39] between the updated head h_θ and original head h_{θ_0}:

$$\mathcal{L}_{\text{KD}} = \frac{T^2}{|D^z_{\text{replay}}|} \sum_{z \in D^z_{\text{replay}}} D_{\text{KL}}\left(\sigma\left(\frac{h_{\theta_0}(z)}{T}\right) \middle\| \sigma\left(\frac{h_\theta(z)}{T}\right)\right) \quad (3)$$

where T is the temperature and σ is softmax. The T^2 factor matches the implementation. Each update independently samples a reference minibatch with replacement for distillation. Distillation is applied only on replay features because the teacher's reference-period predictions represent its source behavior. On target samples, the teacher's predictions may already reflect absorber-collapse errors, making them unsuitable as distillation targets.

E. Combined Objective and Algorithm

The full training loss is:

$$\mathcal{L} = \mathcal{L}_{\text{ft}} + \lambda_{\text{KD}} \cdot \mathcal{L}_{\text{KD}} \quad (4)$$

V. Experiments

This section evaluates CARE through overall repair performance and baseline comparisons, internal mechanisms, cross-setting stress tests, and diagnostic detection quality. We first describe the primary datasets, evaluation protocols, metrics, and implementation settings; the distinct IoT household protocol is specified with its results.

A. Experimental Setup

Datasets. We use CESNET-TLS-Year22 [20] (178 classes, 12 months, TLS traffic) as the primary benchmark and CESNET-QUICEXT-25 XS [40], [41] as a supporting cross-protocol stress test. Our experiments use monthly DataZoo partitions for both datasets and evaluate drift at month-level granularity; they do not replay flows in timestamp order. For TLS-Year22, we train on M-2022-3 and evaluate at M7, M9, M11, and M12. For QUICEXT-25, we train on M-2024-6 and evaluate on M-2025-5.

Dataset-specific collection change. CESNET-TLS-Year22 has a known distribution shift at week 10 due to a flow exporter update [20]. The change is shared by all methods evaluated on the same chronological split, but it remains a dataset-specific source of shift that limits causal attribution to application evolution alone.

Baselines. We compare five categories: (1) *Unsupervised TTA* (0 labels): Tent [14], EATA [15], SAR [16], CoTTA [23], NOTE [21]. (2) *Self-evolving pseudo-label adaptation* (0 human labels): a Chen-inspired full-model baseline [13] (§V-B2). (3) *Standard active learning* (1,000 labels + CARE components): entropy, coreset [18], random. (4) *Continual learning / drift adaptation* (1,000 labels): MEMENTO [27], ILETC [26], CADE [35], expanded head (§V-B3). (5) *CARE configurations* (1,000 labels): margin and BADGE [19] with all-class replay.

Evaluation protocol. Under the main *strict* protocol, active methods query from the full target pool and the selected samples are excluded before computing metrics. The separate matched-holdout protocol first reserves a fixed seed-wise 20% test subset, uses the remaining 80% only for adaptation, and evaluates the predefined 12 M12 collapse classes on the untouched holdout. Static and TTA rows are single-checkpoint evaluations. Main active-learning and CARE rows use the same fixed source checkpoint and report mean $\pm$ sample standard deviation over five run seeds, which control stochastic selection where applicable, replay sampling, and head-update shuffling; they are not independently trained source encoders. Multi-period, prototype, and architecture studies use three run seeds as noted. The separate retraining audit uses three source-model training seeds and three repair seeds per model.

Metrics. (1) Overall macro-F1 (all 178 classes); (2) collapse-class macro-F1 (12 classes with recall <0.1 in the static model at M12); (3) stable-class macro-F1, the unweighted mean of global per-class F1 over a fixed 20-class post-hoc diagnostic set selected by the smallest absolute M4-to-M12 F1 change among classes with at least 100 M12 instances and nonzero M4 support[1]; this endpoint-defined set measures preservation and does not imply high recall in every intervening month; (4) number of classes in the predefined collapse group that remain below 0.1 recall (12 classes for TLS and three for the QUIC stress test); this is not a count of newly collapsed classes outside that group; (5) per-class *recovery status*: recovered ($R \geq 0.30$), partial ($0.05 \leq R < 0.30$), or failed ($R < 0.05$). We assign these recovery statuses from each class's mean recall across repair seeds and report counts separately for each fixed

[1]Class IDs: 8, 15, 44, 57, 59, 62, 64, 76, 94, 98, 99, 107, 113, 119, 128, 130, 131, 132, 144, and 145.

selector. Tables abbreviate the last three aggregate metrics as Col. F1, Stb. F1, and #Col., respectively.

CARE configuration. Budget B=1,000, all-class replay with k=5 per class (890 features), target repeat factor 2, λ_{KD}=0.5, T=2.0, head fine-tuning learning rate 10^{-3}, and 30 epochs. The default monitoring/replay reference is M4; the classifier is trained on M3.

Collapse-definition sensitivity. Across all tested thresholds and support settings, CARE preserves substantial collapse-class recovery: the identified set varies from 10 to 13 classes, yet the collapse-F1 gain remains Δ=0.208–0.246. Support thresholds of 50 and 100 yield identical results, while s=200 removes only the borderline class 38 at τ=0.10 and 0.15. These results show that the main conclusion is robust to the tested collapse definitions; broader threshold ranges remain for future validation.

B. Overall Performance

Overall performance is assessed in four stages: the primary strict query-excluded evaluation at M12, a seed-matched holdout comparison with self-evolving adaptation, comparisons with continual-learning baselines, and performance across drift periods.

1) Primary Results under Strict Query-Excluded Evaluation: Table IV presents the main comparison at M-2022-12 (nine months after training).

TABLE IV
MAIN RESULTS ON CESNET-TLS-YEAR22 AT M-2022-12. STATIC/TTA ROWS ARE SINGLE-CHECKPOINT EVALUATIONS WITHOUT LABELS. ACTIVE ROWS USE B=1,000 LABELS, M4 ALL-CLASS REPLAY (k=5, 890 FEATURES), KD (λ=0.5), STRICT QUERY EXCLUSION, AND FIVE SEEDS.

Method	Macro	Col. F1	Stb. F1	#Col
No labeled target data:				
Static	0.629	0.028	0.903	12
Tent	0.631	0.026	0.903	12
EATA	0.629	0.028	0.903	12
SAR	0.629	0.026	0.901	11
CoTTA	0.630	0.027	0.903	12
NOTE	0.630	0.027	0.902	12
Active (B=1000, all-class replay+KD, 5 seeds):				
Entropy	0.628±0.004	0.078±0.015	0.887±0.004	10.2
Coreset	0.633±0.005	0.116±0.042	0.886±0.007	8.8
Random	0.668±0.004	0.188±0.024	0.892±0.006	8.2
CARE-Margin	0.686±0.001	0.256±0.009	0.892±0.003	5.8
CARE-BADGE	**0.700±0.003**	**0.332±0.040**	**0.894±0.004**	**4.0**

CARE-BADGE has the highest mean macro-F1 and collapse-class F1 among the repair rows in Table IV, raising macro-F1 to 0.700±0.003 and collapse-class F1 to 0.332±0.040 while retaining stable-class F1 of 0.894. CARE-Margin provides a lower-cost operating point with macro-F1 of 0.686±0.001 and collapse F1 of 0.256±0.009. By contrast, the five tested label-free TTA variants leave collapse F1 below 0.031, and generic entropy and coreset selectors trail random, Margin, and BADGE under the same replay-and-distillation pipeline. These results support the value of verified target labels and appropriate query selection for collapse repair.

Selector-level statistical comparison. Paired t-tests on the strict main experiment provide exploratory evidence for Margin over Random (macro-F1 p=0.00075; collapse F1 p=0.00262) and BADGE over Margin (macro-F1 p=0.00112; collapse F1 p=0.0177). The corresponding exact Wilcoxon signed-rank tests are not significant at n=5 (minimum attainable two-sided p=0.0625), so these rankings are reported with effect sizes and seed-wise variance rather than treated as definitive.

2) Seed-Matched Holdout Comparison with Self-Evolving Adaptation: For a direct comparison with self-evolving adaptation, we conducted a separate seed-matched experiment on the same 1,323,229-instance M12 target pool. This is a Chen-inspired pseudo-label baseline, not an exact reproduction of Chen et al. [13]. Starting from the same source checkpoint (`tls22_cnn/best_model.pt`), each seed uses `RandomState(seed+42)` to split the target pool into a 1,058,584-instance adaptation subset (80%) and a 264,645-instance untouched holdout (20%). Adaptation samples whose maximum softmax confidence meets or exceeds $p \in \{0.90, 0.95, 0.99, 0.997\}$ receive argmax pseudo-labels. We update the encoder and head with AdamW (learning rate 0.0025, weight decay 10^{-4}, batch size 500, 50 epochs), without human target labels, replay, or KD. All values in Table V are computed exclusively on the untouched holdout. We show the paper-aligned $p \geq 0.997$ row and the post-hoc setting with the highest overall macro-F1, $p \geq 0.90$; the latter was not selected on an independent validation set and should not be read as a preregistered operating point.

On the common untouched holdouts, CARE-BADGE achieves the highest mean macro-F1 (0.701) and collapse F1 (0.310). The best-macro Chen-style sensitivity setting reaches 0.614 and 0.014, respectively, and leaves all 12 classes below the collapse threshold. This establishes a large repair advantage on identical seed-wise holdouts, although the methods use different supervision and update budgets: the Chen-style implementation uses high-confidence pseudo-labels and full-model fine-tuning, whereas CARE uses 1,000 verified target labels, 890 reference features, and head-only repair. Within CARE, Margin exceeds Random after Holm correction; BADGE has higher mean macro-F1 and collapse F1 than Margin, but five seeds do not resolve that selector-level difference.

3) Comparison with Continual Learning Baselines: To situate CARE against continual-learning and drift-adaptation methods, we adapt four representative approaches to our collapse-repair setting (Table VI). All labeled methods use B=1,000, strict query exclusion, and five seeds. Except for selector-specific CADE-score and CARE-BADGE rows, labeled baselines use the same Margin-selected batch. Except for MEMENTO (which fine-tunes the full model), all methods share the same frozen encoder and differ in how the classification head is updated or what replay data is used.

- **MEMENTO** [27]: Full-model fine-tuning (encoder + head) with KD on all-class replay. MEMENTO's output rectification reduces to a no-op when replaying all classes, so we report the core mechanism only.
- **ILETC** [26]: A conditional Wasserstein generative adversarial network with gradient penalty (WGAN-GP) generates synthetic replay features per class; the head

TABLE V
SEED-MATCHED 20% HOLDOUT COMPARISON AT M12 (MEAN ± SAMPLE STANDARD DEVIATION OVER 5 SEEDS). HUMAN LABELS COUNT UNIQUE QUERIED TARGET LABELS; PSEUDO-LABELS ARE THE MEAN HIGH-CONFIDENCE TARGET SAMPLES USED FOR OUR CHEN-STYLE FULL-MODEL FINE-TUNING. PROGRAM WALL TIME IS MEASURED ON ONE NVIDIA L20 UNDER EACH METHOD'S FINAL CONFIGURATION; DASHES DENOTE CONFIGURATIONS NOT INCLUDED IN THE TIMING RUN.

Method	Human	Pseudo	Macro F1	Collapse F1	Stable F1	#Col.	Wall (s)
Static	0	0	0.6308±0.0020	0.0313±0.0055	0.9019±0.0015	11.8	–
Chen-style ($p \geq 0.997$)	0	298.8K	0.5050±0.0097	0.0008±0.0018	0.8063±0.0208	12.0	80.0±1.0
Chen-style ($p \geq 0.90$, sensitivity)	0	628.7K	0.6140±0.0058	0.0136±0.0035	0.8887±0.0080	12.0	136.4±0.5
CARE-Entropy	1,000	0	0.6316±0.0025	0.0797±0.0097	0.8863±0.0039	10.0	–
CARE-Random	1,000	0	0.6722±0.0021	0.2246±0.0370	0.8934±0.0039	7.0	–
CARE-Margin	1,000	0	0.6885±0.0022	0.2835±0.0185	0.8932±0.0047	**4.6**	48.5±0.8
CARE-BADGE	1,000	0	**0.7008±0.0068**	**0.3099±0.0473**	**0.9004±0.0042**	4.8	166.6±8.2

is fine-tuned on target labels + generated replay without knowledge distillation.

- **CADE-score** [35]: A contrastive autoencoder computes per-sample drift scores; the top-B highest-scoring samples are selected for labeling instead of margin sampling. Repair uses the same head fine-tuning + replay + KD pipeline as CARE.
- **Expanded head** (inspired by PRIME [29]): The single-layer head (Linear(256, C)) is replaced by a two-layer head (Linear(256, 512)→rectified linear unit (ReLU)→Linear(512, C)), initialized via a dual-channel decomposition that reproduces the original output before fine-tuning. PRIME targets new classes; we evaluate capacity expansion for fixed-class repair.

TABLE VI
CONTINUAL-LEARNING AND DRIFT-ADAPTATION BASELINES AT M-2022-12 (B=1,000, STRICT EVAL, 5 SEEDS). CADE-SCORE AND CARE-BADGE USE THEIR NAMED SELECTORS; OTHER LABELED ROWS USE MARGIN. CARE ROWS ARE REPRODUCED FROM TABLE IV.

Method	Macro	Col. F1	Stb. F1	#Col
Static	0.629	0.028	**0.903**	12.0
CL / drift baselines (same label budget):				
CADE-score [35]	0.637±0.004	0.118±0.015	0.889±0.003	9.0
ILETC-GAN [26]	0.650±0.004	0.219±0.010	0.851±0.009	5.4
ILETC-real	0.659±0.003	0.236±0.010	0.854±0.016	4.8
Expanded head	0.673±0.003	0.307±0.005	0.871±0.008	4.8
MEMENTO [27]	0.678±0.002	0.255±0.005	0.892±0.004	6.0
CARE-Margin	0.686±0.001	0.256±0.009	0.892±0.003	5.8
CARE-BADGE	**0.700±0.003**	**0.332±0.040**	0.894±0.004	**4.0**

Three findings emerge:

- **Full-model fine-tuning does not improve this operating point.** Under this 1,000-label budget and implementation, MEMENTO achieves comparable collapse F1 to CARE-Margin (0.255 vs. 0.256) but lower macro-F1 (0.678 vs. 0.686), supporting the head-only design for the evaluated setting.
- **Under this implementation, GAN-generated replay trails real replay.** ILETC-GAN underperforms ILETC-real on macro-F1 (0.650 vs. 0.659) and collapse F1 (0.219 vs. 0.236), and both trail CARE-Margin. The experiment does not isolate why. Their larger stable-class degradation (0.851–0.854 vs. 0.892) is consistent with omitting knowledge distillation, although this attribution is not isolated.
- **Capacity expansion helps collapse but hurts stability.** The expanded head reaches 0.307 collapse F1 but 0.871 stable-class F1 and 0.673 macro-F1. CARE-BADGE has higher mean macro-F1 and collapse F1 among the evaluated repair rows, limiting the decrease in stable-class F1 relative to the static baseline to 0.009.

CADE-score has the lowest collapse F1 among the repair rows in Table VI (0.118); this result is consistent with the hypothesis that anomaly-oriented scores need not select boundary-informative samples, but the mechanism was not isolated. Overall, CARE-BADGE has the highest mean macro-F1 and collapse F1 among the evaluated repair methods.

4) Multi-Period Generalization: Table VII shows CARE's effectiveness across four time points with increasing drift severity.

TABLE VII
MULTI-PERIOD RESULTS (MARGIN, B=1000, ALL-CLASS REPLAY k=5, 3 SEEDS). THE FIXED COLLAPSE SET IS THE 12 CLASSES IDENTIFIED BY THE M12 STATIC MODEL.

	Static		CARE		
Period	Macro	Col.	Macro	Col.	Δ Macro
M7 (light)	0.740	0.503	0.776±0.003	0.633±0.028	+0.036
M9 (medium)	0.691	0.280	0.739±0.003	0.392±0.005	+0.048
M11 (heavy)	0.649	0.145	0.708±0.002	0.408±0.016	+0.060
M12 (severe)	0.629	0.028	0.686±0.002	0.255±0.012	+0.057

CARE improves both macro-F1 and collapse F1 at every evaluated drift level. Macro-F1 gains range from +0.036 at M7 to +0.060 at M11, while collapse F1 reaches 0.633 at M7 and remains 0.255 even under severe M12 drift. This consistent pattern demonstrates robustness across drift severity and indicates that earlier maintenance can offer stronger recoverability; sequential interventions remain to be tested.

C. Internal Mechanism Analysis

This subsection analyzes how the main components and design choices contribute to repair, covering component variants, sample-selection strategies, and replay strategies.

1) Component-Variant Study: Table VIII compares seven implementation variants, summarized in Figure 4. All use Margin selection, strict evaluation, and five seeds, but the

study is not fully factorial: FT-only/FT+KD use target repeat 1, FT+Rpl/Full CARE use repeat 2, and FT+KD uses 150 targeted replay features rather than 890 all-class features. The rows compare evaluated bundles rather than isolating single-component effects.

TABLE VIII
COMPONENT VARIANTS AT M12 (MARGIN, STRICT EVALUATION, 5 SEEDS). TARGETED-150 CONTAINS STABLE+ABSORBER FEATURES; ALL-890 CONTAINS FIVE FEATURES PER CLASS. FT, RPL, KD, AND REP. DENOTE FINE-TUNING, REPLAY, KNOWLEDGE DISTILLATION, AND TARGET REPEAT.

Config.	Target B	Replay	KD	Rep.	Macro	Col. F1	Stb. F1	#Col
Static	0	None	–	–	0.629	0.028	**0.903**	12
Replay only	0	All-890	–	–	0.563±0.007	0.044±0.019	0.816±0.026	10.2
Rpl+KD	0	All-890	✓	–	0.618±0.005	0.039±0.016	0.888±0.009	10.8
FT only	1000	None	–	1	0.638±0.000	0.134±0.001	0.839±0.001	7.0
FT+KD	1000	Targeted-150	✓	1	0.670±0.000	0.152±0.007	0.887±0.004	7.8
FT+Rpl	1000	All-890	–	2	0.659±0.003	0.236±0.010	0.854±0.016	4.8
Full CARE	1000	All-890	✓	2	**0.686±0.001**	**0.256±0.009**	0.892±0.003	5.8

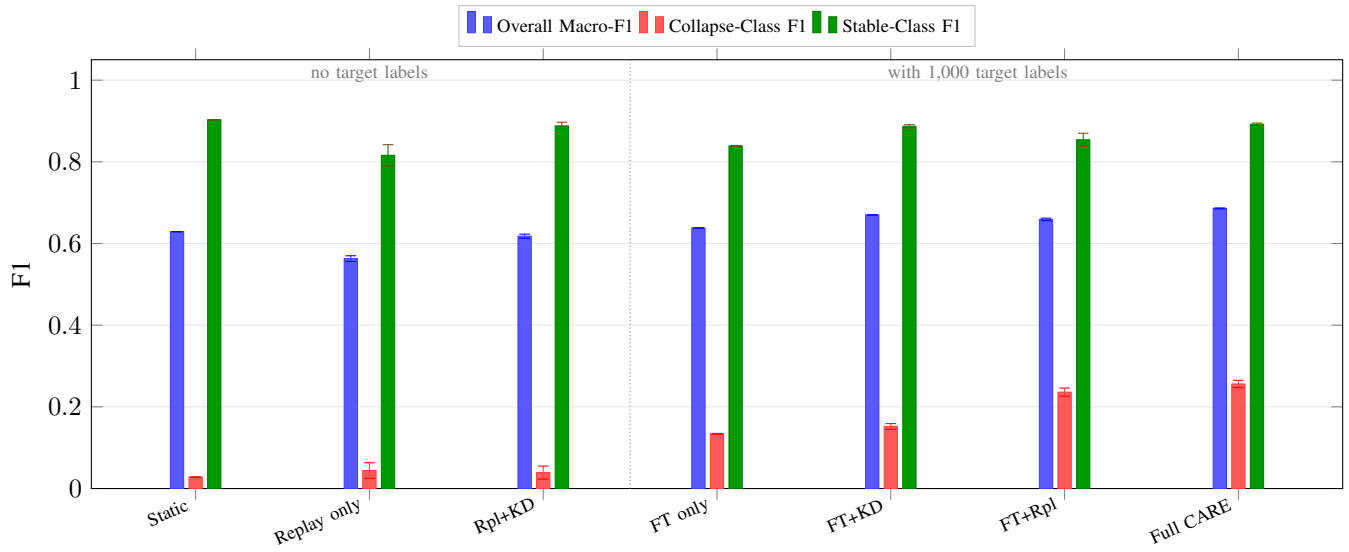


Fig. 4. Original component bundles in Table VIII. Full CARE has the highest macro-F1 and collapse F1 within this study. Typeset from the archived values with Codex-assisted LaTeX layout adjustments.

Without target labels, replay-only rows leave collapse F1 below 0.05; adding distillation improves stable F1 from 0.816 to 0.888 but does not repair collapse. With target labels:

- **FT only** raises collapse F1 from 0.028 to 0.134 but lowers stable F1 from 0.903 to 0.839.
- **FT+KD** restores stable F1 to 0.887, with collapse F1 of 0.152.
- **FT+Rpl** reaches 0.236 collapse F1 and 0.659 macro-F1, with stable F1 of 0.854.
- **Full CARE** reaches 0.686 macro-F1, 0.256 collapse F1, and 0.892 stable F1.

The evaluated bundles associate target supervision with collapse recovery and reference-based regularization with stable-class preservation. Because replay coverage and target repetition also vary, Table VIII does not isolate each loss term. Supplementary Material S2 reports additional fixed-budget, matched-update controls on independently reconstructed public rows. Reference distillation without reference cross-entropy gives higher mean F1 and lower negative-flip rates than full CARE in those controls; full CARE avoids additional threshold-defined collapses in two BADGE seeds. These results qualify the incremental value of reference supervision, without replacing the original S-split results.

Fine-tuning depth. We compare head-only repair (default) with full-model fine-tuning (encoder + head, learning rate scaled by $0.1\times$) under the same 1,000-label budget (Margin, all-class replay k=5, 5 seeds). Full fine-tuning achieves macro-F1 of 0.678 ± 0.001 and collapse F1 of 0.256 ± 0.005, while head-only repair reaches 0.686 ± 0.001 and 0.256 ± 0.009, respectively. Collapse recovery is comparable, but full fine-tuning slightly reduces overall and stable-class performance under the same label budget; this may reflect limited supervision for updating the encoder's $\sim$200K parameters. The result supports freezing the encoder when the operational objective is a bounded repair under a small label budget.

2) Selection Strategy Comparison:

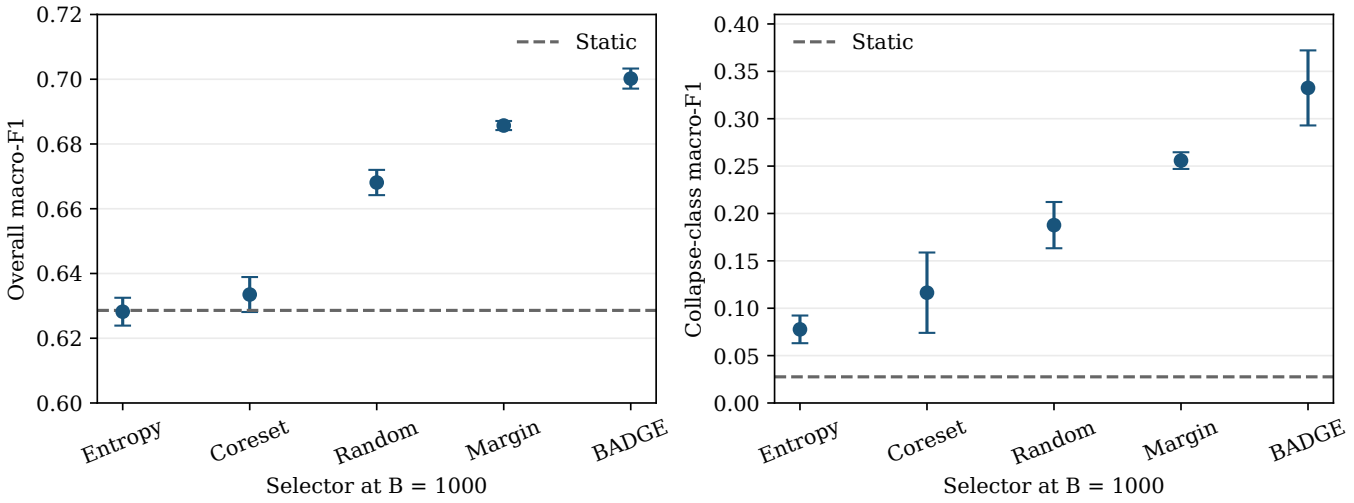


Fig. 5. Selector comparison at B=1,000 with the same all-class replay and distillation pipeline (strict query exclusion; mean $\pm$ sample standard deviation over five repair seeds). Dashed lines show the static model. The plotted values correspond to Table IV; mean differences do not establish a definitive selector ranking. Replotted from archived numeric records using a Codex-assisted Matplotlib script.

The selector comparison is shown in Figure 5. Margin sampling selects globally low-margin samples, which may include absorber–victim boundaries. BADGE [19] achieves the highest collapse F1 (0.332 ± 0.040), 30% above Margin (0.256 ± 0.009), and macro-F1 (0.700 ± 0.003 vs. 0.686 ± 0.001). BADGE has the higher mean performance when its selector complexity is acceptable, but the protocol-matched benchmark makes the trade-off explicit: CARE-BADGE is the slowest timed configuration at 166.6 ± 8.2 s, about $3.4\times$ the 48.5 ± 0.8 s required by CARE-Margin.

With a fixed selector (Table IX), Margin yields 4 recovered, 3 partial, and 5 failed classes, whereas BADGE yields 5 recovered, 5 partial, and 2 failed classes, using seed-mean recall under the strict protocol. Both selectors allocate very few labels to collapse classes (0–13.6 per class on average out of 1,000). These cases indicate that recovery can arise from global boundary updates driven by target labels outside the affected class together with all-class replay, rather than direct labels for every collapsed class.

TABLE IX
PER-CLASS RECOVERY AT M-2022-12 (B=1,000, ALL-CLASS REPLAY k=5, STRICT EVAL, 5 SEEDS; MEAN ± SAMPLE STANDARD DEVIATION). SELECTED = MEAN TARGET LABELS ALLOCATED TO THAT CLASS.

			Static		CARE-Margin		CARE-BADGE		Selected	
Class	Abs.	n	Recall	F1	Recall	F1	Recall	F1	Marg	BADGE
56	96	12137	0.000	0.001	0.722±0.033	0.784±0.008	**0.793±0.017**	**0.829±0.021**	12.0	13.6
163	46	1741	0.016	0.026	0.143±0.031	0.168±0.024	**0.226±0.140**	**0.235±0.123**	3.0	4.8
174	2	1507	0.000	0.000	**0.195±0.005**	**0.262±0.012**	0.029±0.040	0.044±0.060	4.0	1.6
48	14	3361	0.001	0.002	0.437±0.009	0.568±0.007	**0.722±0.234**	**0.767±0.146**	4.0	4.8
38	45	193	0.052	0.090	**0.656±0.095**	**0.410±0.057**	0.242±0.277	0.210±0.210	3.0	1.2
69	105	1866	0.005	0.008	0.031±0.017	0.042±0.021	**0.157±0.203**	**0.177±0.219**	3.0	2.4
104	2	1744	0.010	0.016	**0.703±0.044**	**0.664±0.031**	0.604±0.154	0.594±0.073	8.0	3.6
47	5	618	0.000	0.000	0.002±0.002	0.004±0.003	**0.404±0.201**	**0.462±0.168**	0.0	2.2
66	71	1955	0.001	0.002	0.001±0.000	0.002±0.001	**0.133±0.118**	**0.144±0.121**	1.0	4.2
10	156	1004	0.098	0.162	0.074±0.030	0.121±0.044	**0.389±0.215**	**0.441±0.193**	0.0	1.8
109	71	393	0.000	0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.0	1.6
26	13	807	0.022	0.025	0.035±0.012	0.046±0.011	**0.125±0.140**	**0.087±0.069**	1.0	2.6
Macro (12 classes)			0.017	0.028	0.250	0.256	**0.319**	**0.332**		

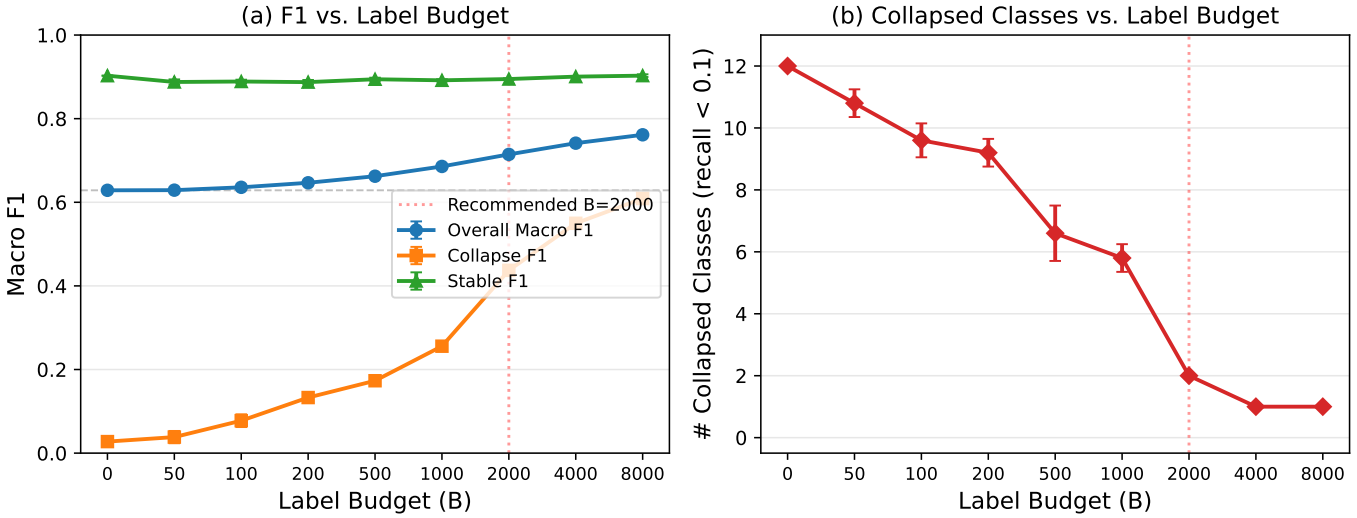


Fig. 6. Label budget vs. performance (margin, 5 seeds, error bars). Left: Overall, collapse, and stable macro F1. Right: number of collapsed classes (recall < 0.1). The knee at B=2,000 reduces collapsed classes from 12 to 2; beyond B=4,000 gains are marginal.

Figure 6 shows performance across label budgets from B=0 to B=8,000. Collapse F1 improves steadily from 0.028 (static) to 0.609±0.007 at B=8,000, while stable-class F1 remains at least 0.887; this limits, but does not eliminate, degradation relative to the 0.903 static stable-class F1. B=2,000 is an empirical knee: it raises collapse F1 to 0.437±0.004 (a 16× improvement over static) and reduces collapsed classes from 12 to 2, although the preferred operating point depends on local annotation cost. Beyond B=4,000 (0.550±0.007), gains are marginal (+0.015 per additional 1,000 labels vs. +0.181 in the 1,000→2,000 range). All labeled CARE, active-learning, and continual-learning comparisons use B=1,000. Chen-style rows use no human labels and a separate pseudo-label regime; the budget curve shows how larger human-label budgets affect CARE.

3) Replay Strategy Comparison: A key design choice is whether to replay all classes or only detected/targeted classes. Table X compares replay strategies under matched conditions (margin, B=1,000, 5 seeds).

All-class replay with k=5 yields the best results (0.686 macro-F1). To assess whether all-class replay benefits only from using more features, we match the total replay budget: targeted replay with k=30 (≈900 features, matched budget) achieves 0.678, while all-class k=1 (178 features) achieves 0.680. This comparison supports class-coverage breadth rather than replay volume as an explanation: anchoring all 178 class

TABLE X
REPLAY STRATEGY COMPARISON (MARGIN, B=1000, 5 SEEDS). "TARGETED" USES AN ORACLE STABLE+ABSORBER LIST; RESULTS SUPPORT CLASS-COVERAGE BREADTH RATHER THAN REPLAY VOLUME AS THE EXPLANATION IN THIS COMPARISON.

Replay Config.	Macro	Col. F1	n_{rpl}
All-class k=5 (890)	**0.686±0.001**	**0.256±0.009**	890
Targeted k=30 (∼900)	0.678±0.001	0.243±0.003	∼900
All-class k=1 (178)	0.680±0.003	0.231±0.015	178

boundaries performs better than concentrating replay on a subset.

This finding also clarifies the role of the detection module. Its intended value in the proposed deployment workflow is *not* to guide replay class selection—all-class replay performs better in the tested ablation—but to (1) flag classes at risk of degradation for operator investigation and (2) indicate *when* repair should be considered by the operator.

Prototype replay. In the tested targeted-replay variant, prototype replay retains 98.7% of individual-exemplar macro-F1 (0.664 ± 0.002 versus 0.673 ± 0.001) while reducing stored features from about 150 KiB to 60 KiB. This result shows that storage-efficient replay is feasible in the tested setting; formal privacy guarantees and validation in the default all-class configuration remain future work.

D. Generalization and Robustness

This subsection examines whether the observed repair behavior persists across model architectures, transport protocols, and source-training and checkpoint variation.

1) Architecture Robustness: To test whether normalization choice alone explains the TTA result, we train an otherwise identical CNN with Batch Normalization (BN): BN-Adapt [42] raises macro-F1 by only 0.009 while collapse F1 stays at 0.032, and NOTE degrades substantially (macro-F1: 0.042). A targeted-replay CARE-Margin variant improves the BN model (0.613→0.665 macro, 0.030→0.314 collapse). We also train a Transformer backbone [43] (4-layer, 4-head, 256-dim). Under the tested targeted-replay CARE-Margin protocol, it reaches 0.767 ± 0.001 macro-F1 and 0.401 ± 0.003 collapse F1. Because this replay protocol differs from the default all-class CNN experiment, we do not treat the gain magnitudes as a protocol-matched backbone comparison.

2) Cross-Protocol Generalization: QUICEXT-25 stress test. To test whether the repair protocol transfers beyond TLS traffic, we evaluate CARE on the CESNET-QUICEXT-25 XS split [40], [41], a 44-class QUIC dataset with monthly partitions. We train on M-2024-6 and evaluate on M-2025-5, where three drift-induced classes (17, 23, and 25) fall below 0.1 recall. Class 21 also has near-zero target recall but is excluded because the model already fails to learn it in the reference period; it is not a drift-induced collapse case.

Table XI reports a label-budget sweep. Static macro-F1 is 0.502 and collapse-class F1 is 0.049. With 1,000 queried target labels, CARE-Margin raises macro-F1 to 0.564 ± 0.003 and collapse F1 to 0.114 ± 0.002, reducing collapsed classes from 3.0 to 2.0. Larger budgets are more effective on QUICEXT-25: CARE-BADGE at B=4,000 reaches 0.586 ± 0.007 macro-F1 and 0.191 ± 0.030 collapse F1, with collapsed classes reduced to 1.2 ± 0.45. Here B denotes the number of queried target labels, not the number of feature vectors used for head training. With target repeat factor 2 and 220 all-class replay features, the B=4,000 setting uses 8,220 head-training feature vectors but only 4,000 newly labeled target samples.

TABLE XI
CROSS-PROTOCOL GENERALIZATION ON CESNET-QUICEXT-25 XS (M-2024-6 → M-2025-5, 44 CLASSES, 5 SEEDS). B IS THE NUMBER OF QUERIED TARGET LABELS; STRICT EVALUATION EXCLUDES THEM.

Method	B	Macro-F1	Collapse F1	#Col.
Static	0	0.502	0.049	3.0
CARE-Margin	1000	0.564 ± 0.003	0.114 ± 0.002	2.0 ± 0.0
CARE-BADGE	1000	0.563 ± 0.006	0.091 ± 0.017	2.8 ± 0.45
CARE-Margin	2000	0.577 ± 0.003	0.126 ± 0.014	2.0 ± 0.0
CARE-BADGE	2000	0.578 ± 0.006	0.144 ± 0.031	2.2 ± 0.84
CARE-Margin	4000	0.578 ± 0.002	0.132 ± 0.021	1.8 ± 0.45
CARE-BADGE	4000	$\mathbf{0.586\pm0.007}$	$\mathbf{0.191\pm0.030}$	$\mathbf{1.2\pm0.45}$

The QUICEXT-25 stress test demonstrates positive cross-protocol transfer: CARE improves both macro-F1 and collapse F1 while reducing the number of collapsed classes. The strongest tested setting, CARE-BADGE with B=4,000, reaches macro-F1 of 0.586 ± 0.007 and collapse F1 of 0.191 ± 0.030, with collapsed classes reduced to 1.2 ± 0.45. The larger label requirement defines a different QUIC operating point rather than a failure of transfer and motivates protocol-aware maintenance budgets.

3) Training-Seed and Checkpoint Robustness: A targeted-replay retraining audit (3×3=9 runs) yields static macro-F1 of 0.620 ± 0.001 and CARE macro-F1 of 0.667 ± 0.007 across training seeds. Our primary checkpoint (static 0.629, best epoch 27/30, validation F1 0.892) is about 0.009 above the audit mean, larger than the audit standard deviation and therefore evidence of checkpoint/run sensitivity. The targeted-replay CARE variant improves over its corresponding static model in all audited training seeds, although the gain magnitude varies.

4) Exploratory IoT Device-Type Stress Test: To examine head repair on an actual IoT label space, we use the public residential IoT flow archive associated with [1]. This mixed-protocol, closed-set task identifies 24 known device types from flow metadata. The official device inventory maps device MAC addresses to type labels; evaluation includes flows from those devices, while MAC addresses, IP addresses, ports, device labels, and timestamps are excluded from model input.

For each home, observed days are ordered and split chronologically into source, query, development, and final windows at approximately 50%, 70%, and 85% of the days. Eleven homes have all 24 types in the source window; the remaining home lacks two source types and is excluded from this fixed-vocabulary comparison. We train a new tabular MLP on source-period flow statistics and protocol indicators, then draw 1,000 uniform query labels per home and repair seed (five overlapping seeds). The ported CARE head update uses those labels and five historical reference features per source class. Target-only head updating, a frozen-feature random forest, and nearest-anchor classification use the same paid labels in

the development comparison. After inspecting development results, we fixed a final-window protocol comparing static, target-only, CARE, and a standard head-weight interpolation control (0.5 source + 0.5 target). The refitted target and CARE heads had to reproduce their complete frozen development predictions before any final label was decoded. The final-window predictions and evaluation rules were then frozen before scoring all 11 homes. This port changes the source architecture, features, query selector, and update schedule relative to the CNN/BADGE primary experiment and evaluates head repair without the monitoring stage.

On the examined development windows, equal-home mean macro-F1 changes from 0.542786 to 0.544238 (+0.145 percentage points), with CARE gains in 6 of 11 homes and mean conditional negative-flip rate of 0.054. Under the same paid-label budget, target-only, frozen-feature random forest, and nearest-anchor controls have equal-home mean macro-F1 of 0.517, 0.482, and 0.456, respectively. Table XII reports the subsequent final-window comparison by home.

The development class-level outcomes reveal a recovery–interference trade-off hidden by the near-zero average gain. Among 17 home–device-type pairs with at least 50 development rows and static recall below 0.1, mean CARE F1 improves for all 17 and mean recall reaches at least 0.1 for 11. Across all 246 home–type pairs present in development, mean F1 improves for 124, falls for 120, and is unchanged for two. Among 75 pairs with static recall at least 0.8 and support at least 50, seven lose more than five F1 points while eight gain more than five. Class-level F1 may also change through precision while recall moves in the opposite direction. Twelve of the 17 low-recall pairs have fewer than five paid positive examples on average. These exploratory development results informed the final-window endpoints; they did not change CARE training.

On the held-out final windows, mean low-recall-class F1 rises from 0.03988 for the static model to 0.13887 for CARE. Equal-home macro-F1 falls from 0.52006 to 0.51239 (-0.767 percentage points), with gains in 5 of 11 homes; the descriptive 95% home-bootstrap interval for the paired mean difference is $[-2.474, 0.756]$ points. Target-only updating reaches 0.48581, while fixed 0.5 source–target head interpolation reaches 0.52858. CARE's conditional negative-flip rate is 0.05908. Across homes, an average of 1.64 device types meet the prespecified low-recall condition (final support $\geq$50 and static recall < 0.1); CARE restores recall to at least 0.1 for an average of 0.78 such types per home. The corresponding stable-recognition set averages 5.55 types per home, of which 0.42 on average lose more than five F1 points under CARE. No home–type pair meets the stricter incident-collapse condition of development static recall $\geq$0.8 followed by final recall < 0.1 at adequate support.

The prespecified descriptive Spearman associations between CARE's home-level macro-F1 change and development-to-final class-share shift, maximum final class share, and paid-positive coverage of low-recall types are 0.027, -0.345, and -0.012, respectively (11, 11, and 10 eligible homes). These indicators show no clear association with the variation in whole-model benefit across households.

E. Diagnostic Detector Evaluation

Signal ablation. Table XIII shows how each signal contributes to detection performance at M12. The detection ground truth uses the same collapse definition as repair evaluation: recall < 0.1 with support ≥ 50, yielding 12 ground-truth collapsed classes. The detector selects the 20 highest-risk classes (k=20).

Count drop and FD provide a strong two-signal baseline (F1 = 0.563). Relative to the four-signal row, adding entropy rise lifts recall from 0.667 to 0.833 and F1 from 0.500 to 0.625.

Detection across drift levels. Table XIV evaluates detection consistency as a diagnostic monitor across four time points.

Recall is $\geq$0.80 at all periods; precision rises from 0.20 at M7 to 0.50 at M12, coinciding with an increase in ground-truth collapsed classes. At M12, the two missed classes are 38 (small support) and 163 (gradual drift).

Detection weight sensitivity. The manual weights achieve the best observed detection F1 of 0.625 without labeled calibration and match the calibrated weights on M12 (precision 0.50, recall 0.83). Equal weights remain competitive at F1 0.563. The close results across manual, equal, and calibrated settings show that the detector is not highly sensitive to exact weighting over the tested range.

TABLE XII
IoT DEVICE-TYPE FINAL-WINDOW HOLDOUT ON 11 HOMES. EACH HOME USES A SEPARATE SOURCE MLP AND 1,000 UNIFORM PAID LABELS PER REPAIR SEED; CARE AND THE FIXED SOURCE–TARGET HEAD-INTERPOLATION CONTROL (BCWI 0.50) AVERAGE FIVE OVERLAPPING SEEDS. MACRO-F1 AVERAGES DEVICE TYPES PRESENT IN THAT HOME'S FINAL WINDOW. DELTAS ARE PERCENTAGE POINTS VERSUS THE STATIC MODEL, COMPUTED BEFORE ROUNDING. HOMES, RATHER THAN SEEDS, ARE INDEPENDENT UNITS.

Home	Static F1	CARE F1	Δ_{CARE} (pp)	BCWI F1	Δ_{BCWI} (pp)
01	0.5895	0.6100	+2.047	0.6106	+2.106
02	0.4623	0.4606	−0.172	0.4915	+2.916
03	0.5201	0.5171	−0.306	0.5421	+2.198
04	0.4327	0.4444	+1.163	0.4539	+2.115
05	0.4498	0.4332	−1.661	0.4636	+1.386
06	0.5470	0.4829	−6.410	0.5109	−3.615
07	0.5536	0.5009	−5.272	0.4603	−9.331
08	0.4930	0.5214	+2.838	0.5429	+4.987
09	0.5564	0.5687	+1.234	0.6095	+5.310
11	0.5553	0.5593	+0.399	0.5511	−0.420
12	0.5609	0.5379	−2.293	0.5780	+1.716
Equal-home mean	0.52006	0.51239	−0.767	0.52858	+0.852

TABLE XIII
DETECTION SIGNAL ABLATION ON M4→M12 (12 GROUND-TRUTH CLASSES, NO TARGET LABELS USED BY THE DETECTOR, k=20). COUNT DENOTES PREDICTION-COUNT DROP; FD DENOTES FRÉCHET DISTANCE.

Signal Combination	Precision	Recall	F1
FD only	0.350	0.583	0.438
Count + FD	0.450	0.750	0.563
Count + FD + margin + confidence	0.400	0.667	0.500
All 5 signals	**0.500**	**0.833**	**0.625**

TABLE XIV
DETECTION ACROSS DRIFT LEVELS (k=20, RECALL<0.1, SUPPORT≥50). GT DENOTES GROUND TRUTH; RECALL IS ≥0.80 THROUGHOUT.

Period	GT	Precision	Recall	F1
M7 (light)	5	0.200	0.800	0.320
M9 (medium)	8	0.350	0.875	0.500
M11 (heavy)	10	0.450	0.900	0.600
M12 (severe)	12	0.500	0.833	0.625

VI. DISCUSSION

This section interprets the empirical findings, outlines their deployment implications, and delineates the study's limitations and directions for future work.

A. Understanding the Efficacy of CARE

Mechanism and stable-class preservation. The original component bundles associate target labels with repair and reference-based regularization with stable-class preservation. Under the original protocol, CARE raises collapse-class F1 from 0.028 to 0.256 with Margin and 0.332 with BADGE, with stable-class F1 of 0.892 and 0.894 versus 0.903 for the static model. The separately reconstructed controls in S2 qualify the incremental role of reference cross-entropy; the fine-tuning-depth comparison in Section V-C1 supports bounded head repair under the evaluated 1,000-label budget.

Why adjacent methods are insufficient. Entropy- and pseudo-label-based TTA receives little corrective signal when an absorber makes confident but wrong predictions, whereas continual-learning and drift-adaptation baselines primarily target new classes, forgetting, or global objectives. CARE couples a collapse-specific diagnostic with verified target labels and preservation constraints. Tested label-free methods leave collapse F1 near zero, and generic active selectors recover less under the same CARE update pipeline. These comparisons support verified labels and preservation-aware maintenance for the evaluated class-repair problem, with supervision and update costs reported separately.

Generalization of the observed effect. CARE improves macro-F1 at every evaluated TLS drift period and remains effective with CNN and Transformer backbones; the QUICEXT-25 stress test extends the positive result to another protocol with a larger label budget. Together, these experiments support class-level repair across the tested encrypted-application settings. The IoT household port adds a device-type label space and flow representation, where low-recall recovery coexists with cross-home variation and whole-model losses. This finding makes candidate-update validation a practical part of class-aware maintenance.

Selector effectiveness and cost. Finally, the choice of active selector determines how effectively the limited label budget is used. BADGE's higher mean performance than Margin is consistent with its selection of diverse, boundary-informative samples across multiple potential collapse regions. However, the BADGE configuration has about $3.4\times$ the program wall time (166.6 ± 8.2 s versus 48.5 ± 0.8 s on one NVIDIA L20), and corrected significance tests do not establish a definitive ranking. Margin therefore remains attractive when latency and simplicity matter, whereas BADGE offers stronger mean recovery when the additional computation is acceptable.

B. Practical Considerations for Deployment

Label acquisition and maintenance scope. CARE requires 1,000 target labels per repair cycle (0.076% of M12 and 0.094% of the matched adaptation pool) plus 890 stored reference features. Depending on local policy, those labels could be obtained from DPI-assisted labeling on a monitored

subnet, exposed SNI metadata, or manual verification using endpoint and domain records. A detector alert should initiate operator review rather than automatic retraining, so annotation is incurred only after a repair cycle is approved. This study did not measure human annotation time or monetary cost.

Operator-reviewed workflow. An illustrative deployment could run CARE as a scheduled offline maintenance job beside the live classifier. The production model would continue serving predictions while the detector aggregates class-health statistics; an approved alert would authorize a limited label budget, after which a candidate classification head could be repaired on a separate machine, validated, versioned, and activated with rollback available. A monthly cycle matches the granularity evaluated here, but shorter monitoring windows, live acceptance gates, and integration with production traffic-analysis systems remain unvalidated.

Detection and operating trade-offs. The detector maintains recall of at least 0.80 across evaluated periods, but M12 precision is 0.50, so screening favors sensitivity and imposes review cost through false alerts. This behavior is appropriate for a diagnostic monitor only if operators validate alerts before labeling and repair; otherwise, false positives could trigger unnecessary work. Once a cycle is approved, active selection and replay can address several collapse regions together, but recovery of every class is not guaranteed. The downstream value for traffic accounting, security triage, and capacity planning must therefore be established in a field study with application-specific error costs.

C. Implications for Long-Term IoT Traffic Analytics

The proposed workflow in Figure 3 places class-health screening, limited label acquisition, head repair, and candidate validation beside a continuously serving traffic classifier. In an IoT environment, the relevant classes may be device types or application services, depending on the monitoring task. The monitor can flag risk from unlabeled traffic, but an operator must verify a class-level failure before labeling or deployment decisions. Periodic head repair is a bounded maintenance action, and per-class review is needed because an aggregate score can hide both recovered and newly harmed categories.

The 11-home holdout sharpens the acceptance criterion: recovery of low-recall device types can coincide with losses in other types and homes. In a practical maintenance cycle, the candidate head should be reviewed for both recovered and harmed classes before activation. The tested home-level shift indicators do not yet identify which households will benefit.

D. Limitations and Future Work

Technical limitations. Head-only repair cannot recover a class if its encoder representation has fully merged with an absorber; selective encoder adaptation or retraining may be required for such cases at higher cost. CARE repairs existing classes and does not address emerging classes, which require complementary class-incremental mechanisms [28]. S2 varies the reference month on one reconstructed December pool; sensitivity on the original S split and independent target periods remains untested. Prototype replay was evaluated only in a smaller targeted-replay variant; neither stored exemplars nor prototypes currently provide a formal privacy guarantee.

External validity and future work. The primary evidence comes from university-network TLS traffic, with QUICEXT-25 as a cross-protocol stress test. The IoT household analysis evaluates head repair for known device types using mixed-protocol flows, a tabular MLP, and uniform query labels; monitoring, BADGE selection, and gateway deployment remain outside that experiment. The five seeds within each home share data and source training. Final windows were scored after the comparison rules were frozen, while development had used the same homes and archive. Future work should test candidate-update acceptance across sites and encrypted-only IoT traffic, measure annotation and review costs, calibrate detector thresholds, and study adaptive maintenance schedules, privacy-preserving replay, and selective encoder adaptation.

VII. Conclusion

Reliable encrypted traffic classification requires tracking failures that aggregate metrics can conceal. We formalize absorber collapse, in which dominant classes absorb predictions from shifted victim classes. On CESNET-TLS-Year22, 12 of 178 classes collapse within nine months. CARE combines diagnostic monitoring with active target-label selection, all-class reference replay, knowledge distillation, and head-only repair to evaluate recovery alongside stable-class preservation.

Under strict query-excluded evaluation, CARE-BADGE raises macro-F1 from 0.629 to 0.700 ± 0.003 and collapse-class F1 from 0.028 to 0.332 ± 0.040 using 1,000 target labels and 890 replay features, while retaining stable-class F1 of 0.894. Tested TTA variants leave collapse F1 below 0.031. On seed-matched holdouts, our Chen-inspired self-evolving implementation leaves all 12 collapsed classes below the recall threshold. Together, these comparisons support using verified target labels for this failure mode under the reported supervision and update budgets.

The component evidence supports evaluating target supervision together with reference-based preservation, while the S2 controls qualify the incremental value of reference cross-entropy. Gains across tested drift periods and backbones, together with the QUIC stress test, support reuse in the evaluated encrypted-application settings. The 11-home device-type holdout adds evidence of low-recall-class recovery and variation across households. CARE thus provides a class-level maintenance framework with strong encrypted-application results and a clear next requirement for heterogeneous IoT settings: validate each candidate update against harm to other classes.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data and Code Availability

The experiments use the public CESNET-TLS-Year22 dataset [20], available at https://doi.org/10.5281/zenodo.10608607, CESNET-QUICEXT-25 [40], [41], available at

https://doi.org/10.5281/zenodo.17249078, and the public residential IoT flow archive associated with [1], available at https://iotanalytics.unsw.edu.au/homedataset.html. Supplementary Material S1 is deposited in Figshare and is accessible for peer review via https://figshare.com/s/bf40e243e93c5b2ac085. S1 provides implementation files, configurations, archived experiment records, and available source checkpoints for the original CESNET experiments. Its result index maps the original CESNET tables and figures to the corresponding records and distinguishes strict query-excluded metrics from full-pool metrics. This Figshare link provides private peer-review access rather than a public DOI.

Raw traffic datasets are distributed through the public repositories cited above and are not repackaged in S1. The original exact size-S row identities and full feature caches are unavailable. The archived materials support verification against the saved experiment records but do not provide a complete end-to-end reproduction of the original experiments. Supplementary Material S2 reports the separate reconstruction controls. Their frozen queries, selected training and reference features, fitted heads, per-class records, and provenance are included under `local_controls/` in S1. These controls use independently reconstructed row identities and a reconstructed class mapping and are not substituted for the original size-S results.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During preparation of this manuscript, OpenAI Codex assisted with language editing, organization of submission materials, reproducible Matplotlib rendering from archived numeric records, and implementation of the IoT holdout evaluation scripts. It also assisted with implementing, executing, and verifying the separate reconstruction controls summarized in Supplementary Material S2. The original CESNET main-table results are retained; the reconstruction controls and IoT port are identified as distinct protocols. The authors are responsible for reviewing the final text, code, and results and for the accuracy and integrity of the article.

References

[1] A. Pashamokhtari, N. Okui, M. Nakahara, A. Kubota, G. Batista, and H. Habibi Gharakheili, "Dynamic inference from IoT traffic flows under concept drifts in residential ISP networks," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 15 761–15 773, 2023.

[2] S. Azizi, F. V. Calderan, N. Okui, M. Nakahara, A. Kubota, M. G. Quiles, G. Batista, and H. Habibi Gharakheili, "Towards label shift adaptation for robust IoT device identification," in *Proceedings of the 7th Joint Workshop on CPS and IoT Security and Privacy*, 2025, pp. 105–110.

[3] S. Azizi, N. Okui, M. Nakahara, A. Kubota, G. Batista, and H. Habibi Gharakheili, "Maintaining IoT device identification under concept drift via budget-aware traffic labeling," 2026, arXiv preprint arXiv:2608.15465. [Online]. Available: https://arxiv.org/abs/2608.15465

[4] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.

[5] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, and Z. Xue, "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 5420–5427. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/25674

[6] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1988–2014, 2019.

[7] P. Wang, X. Chen, F. Ye, and Z. Sun, "A survey of techniques for mobile service encrypted traffic classification using deep learning," *IEEE Access*, vol. 7, pp. 54 024–54 033, 2019.

[8] D. Soukup, R. Plný, D. Vašata, and T. Čejka, "Drift-based dataset stability benchmark," arXiv preprint, 2025.

[9] N. Malekghaini, E. Akbari, M. A. Salahuddin, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, and S. Tuffin, "Deep learning for encrypted traffic classification in the face of data drift: An empirical study," *Computer Networks*, vol. 225, p. 109648, 2023.

[10] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[11] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[12] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.

[13] Z. Chen, G. Cheng, J. Li, T. Qin, Y. Zhou, and X. Luan, "Drift-oriented self-evolving encrypted traffic application classification for actual network environment," arXiv preprint, 2025.

[14] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=uXl3bZLkr3c

[15] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan, "Efficient test-time model adaptation without forgetting," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 16 888–16 905. [Online]. Available: https://proceedings.mlr.press/v162/niu22a.html

[16] S. Niu, J. Wu, Y. Zhang, Z. Wen, Y. Chen, P. Zhao, and M. Tan, "Towards stable test-time adaptation in dynamic wild world," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=g2YraF75Tj

[17] B. Settles, "Active learning literature survey," University of Wisconsin–Madison Department of Computer Sciences, Computer Sciences Technical Report 1648, 2009. [Online]. Available: https://research.cs.wisc.edu/techreports/viewreport.php?report=1648

[18] O. Sener and S. Savarese, "Active learning for convolutional neural networks: A core-set approach," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=H1aIuk-RW

[19] J. T. Ash, C. Zhang, A. Krishnamurthy, J. Langford, and A. Agarwal, "Deep batch active learning by diverse, uncertain gradient lower bounds," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=ryghZJBKPS

[20] K. Hynek, J. Luxemburk, J. Pešek, T. Čejka, and P. Šiška, "CESNET-TLS-Year22: A year-spanning TLS network traffic dataset from backbone lines," *Scientific Data*, vol. 11, no. 1, p. 1156, 2024.

[21] T. Gong, J. Jeong, T. Kim, Y. Kim, J. Shin, and S.-J. Lee, "NOTE: Robust continual test-time adaptation against temporal correlation," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 253–27 266. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/ae6c7dbd9429b3a75c41b5fb47e57c9e-Abstract-Conference.html

[22] K. Jerabek, J. Luxemburk, R. Plny, J. Koumar, J. Pesek, and K. Hynek, "When simple model just works: Is network traffic classification in crisis?" arXiv preprint, 2025.

[23] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 7201–7211. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/html/Wang_Continual_Test-Time_Domain_Adaptation_CVPR_2022_paper.html

[24] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016. [Online]. Available: https://jmlr.org/papers/v17/15-239.html

[25] M. Long, Z. Cao, J. Wang, and M. I. Jordan, "Conditional adversarial domain adaptation," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018, pp. 1640–1650. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/hash/ab88b15733f543179858600245108dd8-Abstract.html

[26] W. Zhu, X. Ma, Y. Jin, and R. Wang, "ILETC: Incremental learning for encrypted traffic classification using generative replay and exemplar," *Computer Networks*, vol. 224, p. 109602, 2023.

[27] F. Cerasuolo, A. Nascita, G. Bovenzi, G. Aceto, D. Ciuonzo, A. Pescapé, and D. Rossi, "MEMENTO: A novel approach for class incremental learning of encrypted traffic," *Computer Networks*, vol. 245, p. 110374, 2024.

[28] W. Cai, C. Hou, M. Cui, B. Wang, G. Xiong, and G. Gou, "Incremental encrypted traffic classification via contrastive prototype networks," *Computer Networks*, vol. 250, p. 110591, 2024.

[29] T. Qin, G. Cheng, Z. Chen, and Y. Zhou, "PRIME: Plasticity-robust incremental model for encrypted traffic classification in dynamic network environments," arXiv preprint, 2025.

[30] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[31] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018.

[32] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "A continual learning survey: Defying forgetting in classification tasks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3366–3385, 2022.

[33] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, and Y. Kalantidis, "Decoupling representation and classifier for long-tailed recognition," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=r1gRTCVFvB

[34] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=37nvvqkCo5

[35] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "CADE: Detecting and explaining concept drift samples for security applications," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2327–2344. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/yang-limin

[36] W. Liu, C. Zhu, Z. Ding, H. Zhang, and Q. Liu, "Multiclass imbalanced and concept drift network traffic classification framework based on online active learning," *Engineering Applications of Artificial Intelligence*, vol. 117, p. 105607, 2023.

[37] R. Gupta, S. Liu, R. Zhang, X. Hu, X. Wang, H. Benkraouda, P. Kommaraju, P. Cao, N. Feamster, and K. Nahrstedt, "Generative active adaptation for drifting and imbalanced network intrusion detection," arXiv preprint, 2025.

[38] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[39] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint, 2015, presented at the NIPS 2014 Deep Learning Workshop.

[40] J. Luxemburk, K. Hynek, and J. Mücke, "CESNET-QUICEXT-25: A year-long QUIC traffic dataset with EXTended attributes," Dataset, 2025, published 2 October 2025. [Online]. Available: https://zenodo.org/records/17249078

[41] J. Mücke, M. Nawrocki, R. Hiesgen, P. Sattler, J. Zirngibl, G. Carle, J. Luxemburk, T. C. Schmidt, and M. Wählisch, "Waiting for QUIC: Passive measurements to understand QUIC deployments," *Proceedings of the ACM on Networking*, vol. 3, no. CoNEXT4, pp. 1–26, 2025, article 41.

[42] S. Schneider, E. Rusak, L. Eck, O. Bringmann, W. Brendel, and M. Bethge, "Improving robustness against common corruptions by covariate shift adaptation," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 11 539–11 551. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/85690f81aadc1749175c187784afc9ee-Abstract.html

[43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017, pp. 5998–6008. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html